CAMPUSCONNECT

Complete application and code guide

A practical guide to the redesigned placement portal, Django REST API, role permissions, student applications, recruiter and TPO decisions, and the flow connecting them.

Inside: Setup, architecture, each app and model, API reference, annotated code excerpts, end-to-end workflows, troubleshooting, and the decision update fixes.

Source reviewed: current workspace, 30 October 2026.

Frontend: index.html.html | Backend: Django + DRF + SimpleJWT | Database: SQLite

Lifecycle

Student account -> browse drive -> apply -> eligibility check -> recruiter/TPO review -> shortlist, finalize/select, or reject -> student tracker refreshes with the decision.

1. What the app does

CampusConnect manages campus recruitment. Students browse opportunities and submit eligible applications. Recruiters publish drives and manage applicant decisions. Admin/TPO users can review applications and update statuses. The database is the source of truth; the browser displays information through API requests.

Capability	Where	Behavior
Identity and roles	accounts	Custom User role: STUDENT, RECRUITER, ADMIN; academic fields on the same user.
Drives	drive	Company, role, compensation, CGPA/backlog criteria, recruiter owner.
Applications	application	Student-drive link with APPLIED, SHORTLISTED, SELECTED, REJECTED state.
Eligibility	ApplicationSerializer	Duplicate, CGPA, and backlog checks when an application is created.
Access control	permissions + viewsets	API checks authenticated role by action; UI visibility is not the security boundary.
Portal	index.html.html	Static page using fetch() and Bearer JWTs; inspector displays request/response details.

Request path

High-level flow

```
Browser fetch()
-> Django route in Project/urls.py
-> JWTAuthentication identifies request.user
-> viewset permission checks the action
-> queryset chooses visible records
-> serializer validates or formats values
-> model/database reads or writes
-> JSON response returns to the page
-> JavaScript redraws status, table, and counts
```

2. Architecture and project files

File	Responsibility
Project/settings.py	Installed apps, middleware, SQLite, JWT authentication, CORS, custom user model.
Project/urls.py	Register/login/refresh, role-zone routes, and DRF router for drives/applications.
accounts/models.py	AbstractUser extension with role, CGPA, backlogs.
accounts/serializers.py	Registration; create_user hashes passwords.
accounts/permissions.py	Role permission checks.
accounts/views.py	Registration and student/recruiter/admin role-zone views.
drive/models.py, serializers.py, views.py	Placement drive data, output representation, CRUD permissions.
application/models.py, serializers.py, views.py	Application data, validation, visibility, decisions.
index.html.html	Redesigned client: login, navigation, drives, pipeline, status tracker, inspector.
requirements.txt	Pinned Python package versions.
db.sqlite3	Local development database.

The browser calls API_BASE = http://127.0.0.1:8000. The HTML is a client, not a Django template. CORS allows the browser to make cross-origin API requests.

Core models

Record	Key fields	Relationship
User	username, password hash, role, cgpa, backlogs	Django AbstractUser plus placement fields.
Drive	company_name, role_title, package_lpa, min_cgpa, max_backlogs, posted_by	One recruiter can post many drives.
Application	student, drive, status, applied_at	One student applies to a drive; unique student-drive pair.

Relationship map

```
User 1 ---- many Drive (posted_by)
User 1 ---- many Application many ---- 1 Drive

Application unique key: (student, drive)
```

3. Setup and run

From the folder containing manage.py, activate the environment, migrate the database, and start Django:

Windows PowerShell

```
.\venv\Scripts\Activate.ps1
python manage.py migrate
python manage.py runserver
```

For a fresh environment:

Fresh install

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

Open index.html.html in a browser. The backend must be reachable at the API_BASE URL in the file. Use the registration form to create an account, then sign in. An admin account for portal decisions means User.role=ADMIN; this custom role is distinct from Django is_staff or is_superuser.

> **Current settings:** SQLite at BASE_DIR/db.sqlite3, DEBUG=True, JWT authentication, and CORS_ALLOW_ALL_ORIGINS=True. These settings are for local development; review them before public deployment.

Useful commands

Django commands

```
python manage.py check
python manage.py showmigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

4. Roles and access rules

Role	What they can do	Backend enforcement
STUDENT	View drives, apply, see only own applications and decisions.	IsStudentRole on create; queryset filters student=request.user.
RECRUITER	View all applications, post/manage drives, shortlist/select/reject.	IsRecruiterRole on drive writes; reviewer permission on application changes.
ADMIN / TPO	View all applications and update candidate decisions.	IsAdminRole on admin zone; reviewer permission on application changes.

Role detection

SimpleJWT login returns access and refresh tokens, but not this custom role field. After login, the frontend probes /api/student-zone/, /api/recruiter-zone/, and /api/admin-zone/. The first successful response determines the real portal role; the dropdown is a fallback if probing fails.

Role guard excerpt

accounts/permissions.py

```
class IsStudentRole(BasePermission):
def has_permission(self, request, view):
# Require a valid login and the matching custom role.
return bool(request.user
and request.user.is_authenticated
and request.user.role == "STUDENT")
```

Application permission routing

application/views.py (simplified)

```
if action == "create":
require IsStudentRole
elif action in ("update", "partial_update", "destroy"):
require IsRecruiterOrAdminRole
else:
require IsAuthenticated
```

5. API reference

Method + path	Purpose and access
POST /api/register/	Public account registration; password is hashed.
POST /api/login/	Verify credentials and issue access/refresh JWT pair.
POST /api/token/refresh/	Exchange refresh token for a new access token.
GET /api/student-zone/	Authenticated STUDENT role check.
GET /api/recruiter-zone/	Authenticated RECRUITER role check.
GET /api/admin-zone/	Authenticated ADMIN role check.
GET /api/drive/	List drives for any authenticated role.
POST /api/drive/	Create drive; recruiter only; server sets posted_by.
GET /api/drive/{id}/	Retrieve drive; authenticated.
PATCH /api/drive/{id}/	Edit drive; recruiter only.
DELETE /api/drive/{id}/	Delete drive; recruiter only; linked applications cascade.
GET /api/application/	Student gets own rows; recruiter/admin gets all.
POST /api/application/	Student only; body contains drive ID; server sets student.
PATCH /api/application/{id}/	Recruiter/admin decision; send status only.
DELETE /api/application/{id}/	Recruiter/admin only.

Apply request

Application creation

```
POST /api/application/
Authorization: Bearer <access-token>
Content-Type: application/json

{ "drive": 3 }

# Server attaches student=request.user.
# New application starts as APPLIED.
```

Decision request

Application status change

```
PATCH /api/application/12/
Authorization: Bearer <access-token>
Content-Type: application/json

{ "status": "SELECTED" }

# Allowed: APPLIED, SHORTLISTED, SELECTED, REJECTED
```

6. Student workflow

- Register as STUDENT and include CGPA/backlogs; authenticate through the login form.
- Browse drives. The page shows role, employer, compensation, eligibility criteria, and application state.
- Click Apply. Browser submits only the drive ID. It never chooses the student ID or initial status.
- Django checks the JWT and IsStudentRole, attaches request.user, and runs eligibility/duplicate validation.
- On success, the database stores APPLIED and the student tracker shows the current status.
- When a recruiter/TPO changes the status, the student application list refreshes every 15 seconds while the page remains open. Refresh List is also available.

Eligibility checks

- CGPA must be greater than or equal to drive.min_cgpa.
- Student backlogs must be less than or equal to drive.max_backlogs.
- A student can have only one application per drive; serializer and database uniqueness both protect this.

Tracker status

Stored value	Meaning shown
APPLIED	Application received.
SHORTLISTED	Passed initial review.
SELECTED	FINALIZED / SELECTED.
REJECTED	Application declined.

Student scope is enforced in ApplicationViewSet.get_queryset(), so changing browser code cannot reveal another student’s application.

7. Recruiter and TPO workflow

- After login, role-zone probes identify the actual backend role.
- Both roles load the full application pipeline. Recruiters and admins can see all applicants for each drive.
- Drive cards calculate applicant counts by matching application.drive to drive.id after normalizing IDs as strings.
- In the candidate table, choose Shortlist, Finalize / Select, or Reject.
- The browser sends PATCH with a status. The backend checks IsRecruiterOrAdminRole and updates the existing application.
- After a successful PATCH, the reviewer reloads applications and the page redraws statuses/counts. Student polling then displays the same stored decision.

Permission implementation

application/views.py

```
class IsRecruiterOrAdminRole(IsRecruiterRole):
def has_permission(self, request, view):
# Accept a recruiter; otherwise check the TPO/admin role.
return (super().has_permission(request, view)
or IsAdminRole().has_permission(request, view))
```

> **Why explicit checks:** Combining instantiated permissions with | caused a TypeError in this project. The explicit class works with the configured DRF version and makes the allowed roles clear.

Reviewer access is not restricted to the recruiter who posted a particular drive. Drive creation/deletion remains recruiter-only in the current API.

8. Browser portal walkthrough

Function / area	What it does
handleRegister	POSTs new account information; stores credentials only through Django registration.
handleLogin	POSTs username/password, stores JWTs in appState, discovers role, opens portal.
callDjangoApi	Adds JSON and Bearer headers, retries after 401 using refresh token, parses response, updates inspector.
fetchDrives	GET drives and draws cards.
fetchApplications	GET role-scoped applications; redraws drive counts, table, overview.
handleApplyDrive	POST application; refreshes application list.
handleUpdateAppStatus	PATCH decision; refreshes application list on success.
renderDrivesGrid	Searches drives, matches application IDs, shows counts for reviewers.
renderApplicationsTable	Applies status filter, renders status/progress, shows reviewer controls only to authorized roles.
buildStepTrackerHtml	Visualizes Applied, Shortlisted, Selected, or Rejected progress.
updateInspectorUI	Shows request status, elapsed time, response data, and permission/validation errors.

Refresh sequence

Data loading

```
refreshAllData()
fetchDrives() -> GET /api/drive/ -> draw cards
fetchApplications() -> GET /api/application/
-> redraw drive applicant counts
-> redraw candidate/student table
-> redraw overview KPIs
```

Student polling calls fetchApplications() every 15 seconds only while a student is logged in. It is a browser refresh, not push messaging; the page must remain open to update automatically.

9. Backend code with comments

Custom user

accounts/models.py

```
class User(AbstractUser):
# Account type selects the portal/API permissions.
role = models.CharField(choices=ROLE, max_length=9, default="STUDENT")
# Used by drive eligibility checks.
cgpa = models.DecimalField(max_digits=4, decimal_places=2, default=0)
backlogs = models.IntegerField(default=0)
```

Secure registration

accounts/serializers.py

```
password = serializers.CharField(write_only=True)

def create(self, validated_data):
# create_user hashes the password; never store raw passwords.
return User.objects.create_user(
username=validated_data["username"],
password=validated_data["password"],
role=validated_data.get("role", "STUDENT"),
cgpa=validated_data.get("cgpa", 0),
backlogs=validated_data.get("backlogs", 0),
)
```

Drive ownership

drive/views.py

```
def get_permissions(self):
# All signed-in roles can browse opportunities.
if self.action in ["list", "retrieve"]:
return [IsAuthenticated()]
# Writes are limited to recruiters.
return [IsRecruiterRole()]

def perform_create(self, serializer):
# Do not trust client-supplied ownership.
serializer.save(posted_by=self.request.user)
```

Application relationship

application/models.py

```
class Application(models.Model):
student = models.ForeignKey(User, on_delete=models.CASCADE)
drive = models.ForeignKey(Drive, on_delete=models.CASCADE)
status = models.CharField(choices=STATUS_CHOICES, default="APPLIED")

class Meta:
# Prevent duplicate student-drive applications in the database.
unique_together = ("student", "drive")
```

9. Backend code with comments (continued)

Application create validation

application/serializers.py

```
def validate(self, data):
# PATCH has an existing instance and may carry status only.
# Do not require drive or rerun create eligibility checks.
if self.instance is not None:
return data

student = self.context["request"].user
drive = data["drive"]

# Block duplicates, under-cutoff CGPA, or excess backlogs.
if Application.objects.filter(student=student, drive=drive).exists():
raise serializers.ValidationError("Already applied.")
if student.cgpa < drive.min_cgpa:
raise serializers.ValidationError("CGPA below cutoff.")
if student.backlogs > drive.max_backlogs:
raise serializers.ValidationError("Too many backlogs.")
return data
```

Application data visibility and permissions

application/views.py

```
def get_queryset(self):
rows = Application.objects.select_related("student", "drive").all()
# Students see only their own applications.
if self.request.user.role == "STUDENT":
return rows.filter(student=self.request.user)
# Recruiter and ADMIN/TPO see the full pipeline.
return rows

def get_permissions(self):
if self.action == "create":
return [IsStudentRole()]
if self.action in ("update", "partial_update", "destroy"):
return [IsRecruiterOrAdminRole()]
return [IsAuthenticated()]
```

The serializer returns student_name, company_name, and role_title as read-only projections from related models. The student foreign key is also read-only in request data; perform_create supplies it securely.

> **PATCH detail:** The update serializer has self.instance, so the short status-only request skips creation checks. This prevents data["drive"] KeyError and lets the status persist.

10. Bug fixes and troubleshooting

Symptom	Cause	Fix / next check
PATCH 500: unsupported operand for |	Permission objects were combined with |.	Use explicit IsRecruiterOrAdminRole checks; server system check passed.
Status-only PATCH fails	Create validation expected a drive for every request.	Only run duplicate/eligibility checks for new application instances.
Student not seeing decision	PATCH may not persist, or student list has not refreshed.	Expect PATCH 200 with new status; student list polls every 15 sec and can be manually refreshed.
Drive applicant count is zero	Strict ID type comparison or cards rendered before applications finished loading.	Compare IDs with String(); redraw drive cards after application fetch.
UI shows wrong role	Custom role absent from standard JWT login response.	Probe role-zone endpoints; do not treat dropdown as backend authority.
403 on apply	Only STUDENT can create.	Verify User.role in database.
403 on status update	Role is not RECRUITER/ADMIN.	Verify actual account role and authenticated token.
400 on apply	CGPA, backlog, or duplicate validation.	Check drive criteria and existing application.
401	Expired/invalid token.	Login again; central helper refreshes access when possible.
500	Unhandled server exception.	Use the first exception and source line in Django traceback.

Expected decision verification

End-to-end check

```
1. Reviewer clicks Select / Finalize.
2. PATCH /api/application/<id>/ returns 200.
3. Response JSON status is SELECTED.
4. Database application status is now SELECTED.
5. Student GET /api/application/ returns that row.
6. Portal renders FINALIZED / SELECTED.
```

11. Current behavior and boundaries

- The development database is SQLite; user, drive, and application records persist across server restarts.
- JWTs are held in JavaScript memory, so a page reload requires the user to log in again.
- Student polling runs every 15 seconds while the student page is open; it is not push notification.
- ADMIN is the portal/TPO role. Django superuser/staff flags alone do not satisfy IsAdminRole.
- Admin/TPO may decide applications, but drive writes remain recruiter-only.
- Deleting a drive cascades to its applications.
- Recruiter/admin application access is college-wide; the code does not limit reviewers to drives they posted.
- Settings have DEBUG enabled and broad CORS for local development; review before deployment.

Endpoint and data summary

Data	Read visibility	Create/update
Drives	Any authenticated role	Recruiter writes; posted_by assigned server side.
Applications	Student: own; reviewer: all	Student create; recruiter/admin update decision.
Status values	Same stored state exposed in API	APPLIED, SHORTLISTED, SELECTED, REJECTED.

Appendix: complete source map

File	What to read there
Project/settings.py	Apps, CORS, DB, custom user, JWT authentication.
Project/urls.py	Public auth, role zones, router routes.
accounts/models.py	User role and academic fields.
accounts/serializers.py	Registration and password hashing.
accounts/permissions.py	Role guard implementations.
accounts/views.py	Register and role-zone API views.
drive/models.py	Drive fields and recruiter owner foreign key.
drive/serializers.py	Drive JSON and read-only recruiter username.
drive/views.py	Drive read/write policy and ownership assignment.
application/models.py	Application statuses and unique student-drive pair.
application/serializers.py	Application response shape and create-only eligibility checks.
application/views.py	Student isolation and reviewer permissions.
index.html.html	Current redesigned UI and browser API client.
requirements.txt	Pinned dependency versions.
db.sqlite3	Local development database.

Reading advice

Start with Project/urls.py to find the route, then follow the view, serializer, and model. For browser behavior, find the corresponding function in index.html.html and follow its call to callDjangoApi. The inspector provides the HTTP evidence; Django logs provide the stack trace for server exceptions.

End of guide